

Universidad Autonoma de Occidente

Modulo 2 — Entrega Final

Integrante	Codigo
Juan Manuel Velazquez Terreros	22501347
Julian Andres Herrera Sanchez	22500247
Juan Sebastian Plazas Gallo	22501580
Juliana Maria Lozano Santa	22500696

Tabla de Contenido

1. Introduccion y Contexto del Proyecto
2. Arquitectura General del Sistema
3. Bloque 1 — Motor RAG con ChromaDB
4. Bloque 2 — Herramienta de Datos Estructurados
5. Bloque 3 — Router Hibrido (HybridRouter)
6. Bloque 4 — Memoria Conversacional
7. Bloque 5 — Interfaz Streamlit con Chat
8. Bloque 6 — Observabilidad con LangSmith
9. Evaluacion y Resultados
10. Conclusiones

1. Introduccion y Contexto del Proyecto

El presente informe documenta el desarrollo del **Modulo 2** del proyecto de investigacion aplicada sobre Manuelita S.A., empresa agroindustrial colombiana fundada en 1864 con operaciones en Colombia, Peru y Chile. Este modulo extiende el sistema Q&A; basico del Modulo 1 hacia un **agente conversacional completo** con memoria, herramientas especializadas y observabilidad.

El objetivo central es construir un agente capaz de responder preguntas sobre Manuelita S.A. de forma precisa, contextual y trazable, diferenciando entre datos exactos (NIT, cifras financieras, directivos) y preguntas narrativas abiertas (cultura, sostenibilidad, estrategia), todo dentro de una interfaz conversacional moderna con historial de conversacion.

Alcance del Modulo 2

Componente	Tecnologia	Estado
Motor RAG	ChromaDB + Embeddings	Completado
Datos Estructurados	JSON en memoria	Completado
Router Hibrido	HybridRouter propio	Completado
Memoria Conversacional	LangChain BufferWindow	Completado
Interfaz Chat	Streamlit	Completado
Observabilidad	LangSmith	Completado

Tabla 1. Componentes del Modulo 2 y su estado de implementacion.

2. Arquitectura General del Sistema

El sistema sigue una arquitectura de capas donde cada capa agrega una capacidad independiente. La figura conceptual a continuacion muestra el flujo completo desde la pregunta del usuario hasta la respuesta generada, pasando por memoria, routing, herramientas y observabilidad.

Capa	Modulo	Responsabilidad
Interfaz	app.py (Streamlit)	Entrada del usuario, burbujas de chat, badges de fuente
Memoria	memory.py	Historial ultimos 5 turnos, deteccion de follow-ups
Agente	agent.py (HybridRouter)	Routing por keywords hacia herramienta correcta
Herramienta 1	tools/structured_tool.py	Datos exactos JSON — 0ms, sin LLM
Herramienta 2	rag_engine.py	Preguntas abiertas — ChromaDB + embeddings + LLM
Observabilidad	langsmith_setup.py	Trazado automatico de todas las llamadas LangChain

Tabla 2. Capas de la arquitectura y sus responsabilidades.

Flujo de Ejecucion

Cuando el usuario realiza una pregunta, el sistema sigue este flujo determinista:

1	La pregunta entra por la interfaz Streamlit y se registra en el historial de sesion.
2	ContextualAgent evalua si la pregunta es un <i>follow-up</i> (referencia a turno anterior). Si lo es, enriquece la consulta con el historial reciente de ConversationBufferWindowMemory.
3	HybridRouter analiza la pregunta: primero busca senales narrativas (<code>_RAG_SIGNALS</code>), luego intenta clasificacion por categoria estructurada. Decide la herramienta.
4	Si la herramienta es Datos Estructurados : consulta el JSON en memoria y retorna la respuesta exacta en ~0ms sin invocar ningun LLM.
5	Si la herramienta es RAG : recupera los k=6 fragmentos mas relevantes del vectorstore ChromaDB y los pasa al LLM (Gemini/Ollama) con el prompt de sistema.
6	La respuesta se guarda en memoria y se renderiza en la UI con un badge de fuente (RAG vs Estructurado) y el tiempo de respuesta.
7	LangSmith registra automaticamente toda la cadena de llamadas en el dashboard en tiempo real.

3. Bloque 1 — Motor RAG con ChromaDB

El motor de **Retrieval-Augmented Generation (RAG)** es la herramienta principal para preguntas abiertas y narrativas sobre Manuelita S.A. Combina búsqueda semantica por vectores con generacion de lenguaje natural para producir respuestas fundamentadas en el corpus documental.

Componentes del Motor RAG

Componente	Detalle
Vectorstore	ChromaDB — base de datos vectorial local persistente
Embeddings Gemini	models/gemini-embedding-001 — 768 dimensiones
Embeddings Local	paraphrase-multilingual-MiniLM-L12-v2 (HuggingFace)
Embeddings Ollama	nomic-embed-text via servidor local
Chunk size	500 caracteres con solapamiento de 80
Recuperacion k	k=6 fragmentos por consulta
Corpus	Archivos Markdown en data_processed/markdown/
Archivo prioritario	key_facts_manuelita.md — formato Q&A estructurado
LLM (Gemini)	gemini-2.0-flash — capa gratuita AI Studio
LLM (local/ollama)	llama3.2:3b via Ollama

Tabla 3. Parametros y componentes del motor RAG.

Prompt de Sistema RAG

El prompt de sistema del motor RAG establece el rol del agente, las reglas de respuesta y los mecanismos anti-alucinacion. Contiene tres directivas criticas:

- **Rol:** Asistente experto en Manuelita S.A. basado exclusivamente en el contexto provisto.
- **Anti-alucinacion:** Si la informacion no esta en el contexto, responder 'No tengo informacion sobre eso en los documentos disponibles'.
- **Soporte conversacional:** Resolver referencias al historial inyectado cuando se detecta un follow-up.

4. Bloque 2 — Herramienta de Datos Estructurados

La herramienta de datos estructurados (**ManuelitaDatosEstructurados**) responde preguntas sobre datos exactos de Manuelita S.A. consultando un archivo JSON en memoria, sin invocar ningun modelo de lenguaje. Esto garantiza respuestas 100% exactas en tiempo constante (~0ms).

Categorias de Datos

Categoria	Ejemplo de pregunta	Dato de ejemplo
nit	Cual es el NIT	891.300.241-7
presidente	Quien es el presidente	Harold Eder Caicedo
fundacion	Cuando fue fundada	1864
países	En que países opera	Colombia, Peru, Chile
empleados	Cuantos empleados tiene	7.971 colaboradores
ingresos	Cuales fueron los ingresos	COP 3.2 billones (2023)
ebitda	Cual fue el EBITDA	COP 612.000 millones
unidades	Cuales son sus unidades	Azucar, Biocombustibles, Frutas...
carbono	Meta de carbono	Net Zero 2050
general	Informacion general	Default RAG si no clasifica

Tabla 4. Categorias de datos estructurados y ejemplos.

El sistema incluye **fuzzy matching** con distancia de Levenshtein (umbral <= 2) para tolerar errores tipograficos en las preguntas, garantizando alta precision incluso con consultas mal escritas. El resultado es siempre 100% de precision en datos exactos independientemente del proveedor LLM configurado.

5. Bloque 3 — Router Hibrido (HybridRouter)

El **HybridRouter** es el componente central de decision del agente. A diferencia de un agente ReAct que depende del LLM para decidir que herramienta usar, el HybridRouter aplica logica determinista por keywords, lo que garantiza routing correcto incluso con LLMs pequenos (llama3.2:3b, gemma:2b) sin capacidad de razonamiento estructurado.

Logica de Decision (Prioridad)

Prioridad	Condicion	Herramienta
1 (mas alta)	<code>_RAG_SIGNALS</code> en la pregunta (valores, cultura, historia, estrategia...)	RAG
2	<code>_detect_category()</code> encuentra categoria exacta (nit, presidente, empleados...)	Datos Estructurados
3 (default)	Ninguna condicion anterior	RAG

Tabla 5. Logica de prioridades del HybridRouter.

El set **_RAG_SIGNALS** contiene terminos narrativos que fuerzan el uso de RAG: valores, cultura, estrategia, historia, mision, vision, proposito, premios, reconocimientos, certificaciones, sostenibilidad, impacto, entre otros. Esta lista puede extenderse facilmente sin modificar la logica central del router.

Resultados de Evaluacion del Router

Proveedor	Routing correcto	Score keywords	Tiempo promedio
gemini-2.0-flash	100%	~95%	1.2s
llama3.2:3b (local)	100%	~60%	8s
llama3.2:3b (ollama)	100%	~65%	6s

Tabla 6. Resultados de evaluacion del HybridRouter por proveedor.

6. Bloque 4 — Memoria Conversacional

La memoria conversacional permite al agente mantener el contexto de los ultimos N turnos de conversacion, resolviendo referencias implicitas como 'Y *que produce alli?*' o '*Cuantos empleados tiene?*' en el contexto correcto de la empresa mencionada anteriormente.

Implementacion Tecnica

Se utiliza **ConversationBufferWindowMemory** de LangChain con una ventana deslizante configurable (default: 5 turnos). La clase **ConversationMemory** actua como wrapper que expone una API simplificada, y **ContextualAgent** extiende ManuelitaAgent integrando la memoria con el motor de herramientas.

Parametro	Valor
Implementacion	ConversationBufferWindowMemory (LangChain)
Ventana por defecto	5 turnos (configurable con MEMORY_WINDOW env var)
Formato del historial	Texto plano: 'Usuario: ... / Asistente: ...'
Deteccion follow-up	Keywords: y, tambien, alli, eso, ese, esa + frases cortas (<5 palabras)
Enriquecimiento	Se inyecta [Contexto previo] al inicio de la pregunta
Persistencia	En memoria de sesion (reset con boton 'Nueva conversacion')

Tabla 7. Parametros de la memoria conversacional.

Resultados del Test de Memoria

Test	Resultado
Memoria basica (save/load)	OK — 6/6 assertions pasadas
Ventana deslizante (window=2)	OK — descarta correctamente turnos antiguos
Deteccion de follow-up	OK — 6/7 correctos (86% >= umbral 85%)
Dialogo completo 6 turnos	OK — routing 100%, follow-ups enriquecidos

Tabla 8. Resultados del test de memoria conversacional (test_memoria_bloque4.py).

7. Bloque 5 — Interfaz Streamlit con Chat

La interfaz de usuario es una aplicacion **Streamlit** con diseno corporativo verde de Manuelita S.A. Implementa un chat conversacional moderno con historial de burbujas, indicadores de fuente por mensaje, y metricas de sesion en tiempo real.

Caracteristicas de la Interfaz

Chat conversacional	Burbujas de mensaje diferenciadas usuario/asistente con scroll automatico
Badges de fuente	Cada respuesta muestra si viene de RAG (verde) o Datos Estructurados (dorado)
Indicador de memoria	Icono de cadena en preguntas de seguimiento enriquecidas con historial
Selector de proveedor	Radio: Local (HuggingFace + Ollama) / Ollama / Google Gemini
Reset de conversacion	Boton 'Nueva conversacion' que borra historial en memoria y UI
Preguntas de ejemplo	8 chips de preguntas predefinidas para demo rapido
Metricas de sesion	Turnos totales, llamadas RAG, llamadas estructuradas, follow-ups
Badge LangSmith	Estado de trazabilidad con link directo al dashboard
Pestanas legacy	Resumen ejecutivo y FAQ del Modulo 1 conservados

Ejecucion de la Aplicacion

La aplicacion se inicia con uno de estos comandos segun el proveedor deseado:

```
# Modo local (sin API, requiere ollama serve):
uv run streamlit run app.py

# Modo Gemini (requiere GEMINI_API_KEY en .env):
$env:LLM_PROVIDER='gemini'; uv run streamlit run app.py

# Verificar LangSmith activo:
uv run python scripts/test_langsmith_bloque5.py
```

8. Bloque 6 — Observabilidad con LangSmith

LangSmith es la plataforma oficial de observabilidad para aplicaciones LangChain. Cuando las variables de entorno **LANGCHAIN_TRACING_V2=true** y **LANGCHAIN_API_KEY** estan configuradas, LangSmith registra automaticamente todas las llamadas al LLM, embeddings, retrievers, herramientas y cadenas sin modificar el codigo del agente.

Componentes de la Integracion

Componente	Descripcion
langsmith_setup.py	Modulo central: init_langsmith(), get_traceable(), badge para Streamlit
@traceable	Decorador en ManuelitaAgent.ask() — run_name='manuelita_ask', tags=['modulo2','hybr
Auto-tracing	LangChain traza automaticamente: LLM, embeddings, retriever, memoria
Badge Streamlit	El sidebar muestra estado activo/inactivo con link al dashboard
test_langsmith_bloque5.py	Script de verificacion — 3 tests: configuracion, traza, metadatos

Tabla 9. Componentes de la integracion LangSmith.

Que se Traza Automaticamente

Componente	Run name en LangSmith	Tipo
ManuelitaAgent.ask()	manuelita_ask	@traceable (explicito)
LLM Gemini	ChatGoogleGenerativeAI	Auto (LangChain)
LLM Ollama	ChatOllama	Auto (LangChain)
Embeddings Gemini	GoogleGenerativeAIEmbeddings	Auto (LangChain)
Embeddings Local	HuggingFaceEmbeddings	Auto (LangChain)
ChromaDB Retriever	VectorStoreRetriever	Auto (LangChain)
Memoria	ConversationBufferWindowMemory	Auto (LangChain)

Tabla 10. Componentes trazados en LangSmith y tipo de registro.

Resultado del Test de Verificacion

El script **test_langsmith_bloque5.py** valida la integracion en 3 etapas y confirmo 3/3 tests OK con LangSmith activo:

- Configuracion: SDK langsmith 0.7.38 instalado, LANGCHAIN_TRACING_V2=true, API key valida
- Traza de agente: pregunta procesada, herramienta correcta, traza registrada en dashboard
- Metadatos: decorador @traceable aplicado, todas las funciones del modulo disponibles

9. Evaluacion y Resultados

El sistema fue evaluado mediante una suite de pruebas integradas que cubre los tres bloques funcionales del agente: herramienta estructurada, router hibrido y memoria conversacional. Los resultados se obtienen ejecutando `test_modulo2_completo.py`, que ejecuta 26 preguntas de evaluacion y genera un reporte JSON automatico.

Resultados Globales por Proveedor

Metrica	gemini-2.0-flash	llama3.2:3b (local)	llama3.2:3b (ollama)
Score datos estructurados	100%	100%	100%
Routing correcto	100%	100%	100%
Score keywords RAG	~95%	~60%	~65%
Deteccion follow-up	86%	86%	86%
Ventana deslizante	OK	OK	OK
Tiempo estructurado	~0ms	~0ms	~0ms
Tiempo RAG promedio	~1.2s	~8s	~6s
LangSmith trazado	Si	Si	Si

Tabla 11. Resumen de evaluacion por proveedor LLM.

Analisis de Resultados

El routing alcanza **100% de precision** con los tres proveedores gracias al HybridRouter deterministico, que no depende del LLM para tomar la decision de herramienta. La herramienta estructurada mantiene **100% de exactitud** y tiempo de respuesta constante (~0ms) independientemente del proveedor.

La calidad de las respuestas RAG varia significativamente entre proveedores: Gemini alcanza ~95% de score en keywords mientras que los modelos locales obtienen entre 60-65%, lo que refleja la mayor capacidad de sintesis y comprension del contexto de gemini-2.0-flash. Para una demo academica de alta calidad se recomienda el proveedor Gemini.

10. Conclusiones

Arquitectura modular y extensible

La separacion en bloques independientes (RAG, herramienta estructurada, router, memoria, UI, observabilidad) permite extender o reemplazar cualquier componente sin afectar los demas. El HybridRouter es particularmente valioso por su independencia del LLM para tomar decisiones de routing.

Routing deterministico como ventaja clave

El HybridRouter logra 100% de routing correcto con cualquier proveedor LLM, superando el enfoque ReAct que falla con modelos pequenos. Esta es la principal innovacion tecnica del modulo respecto a enfoques genericos de LangChain.

Memoria conversacional efectiva

La implementacion de ConversationBufferWindowMemory con ventana deslizante y deteccion de follow-ups permite dialogos naturales multiturnos. El 86% de deteccion de preguntas referenciales supera el umbral academico del 85%.

Observabilidad como practica profesional

La integracion de LangSmith establece una practica de MLOps desde la etapa de prototipo. La trazabilidad completa de cada llamada al LLM, embeddings y retriever facilita la depuracion, optimizacion y auditoria del sistema.

Trabajo futuro

Se identifican como areas de mejora: (1) aumentar el corpus RAG con documentos mas recientes, (2) implementar re-ranking de fragmentos recuperados, (3) agregar autenticacion multi-usuario con memorias independientes por sesion, y (4) desplegar en la nube con LangSmith en modo produccion.

Universidad Autonoma de Occidente · Maestria en IA y Ciencia de Datos · Mayo 2026

Juan Manuel Velazquez Terreros (22501347) · Julian Andres Herrera Sanchez (22500247) · Juan Sebastian Plazas Gallo (22501580) ·
Juliana Maria Lozano Santa (22500696)